

Github task 2

1) install git

Git —version (to check git downloaded)

```
waseemakram@waseem myproject.git % git --version  
git version 2.51.2  
waseemakram@waseem myproject.git %
```

2) Create a repo in github with README.md and .ignore file.

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 General

Owner *

 Waseema761 ▾

Repository name *

git-practice

✓ git-practice is available.

Great repository names are short and memorable. How about **fluffy-octo-succotash**?

Description

0 / 350 characters

2 Configuration

Choose visibility *

Choose who can see and commit to this repository

 Public ▾

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

On ☒

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore ▾

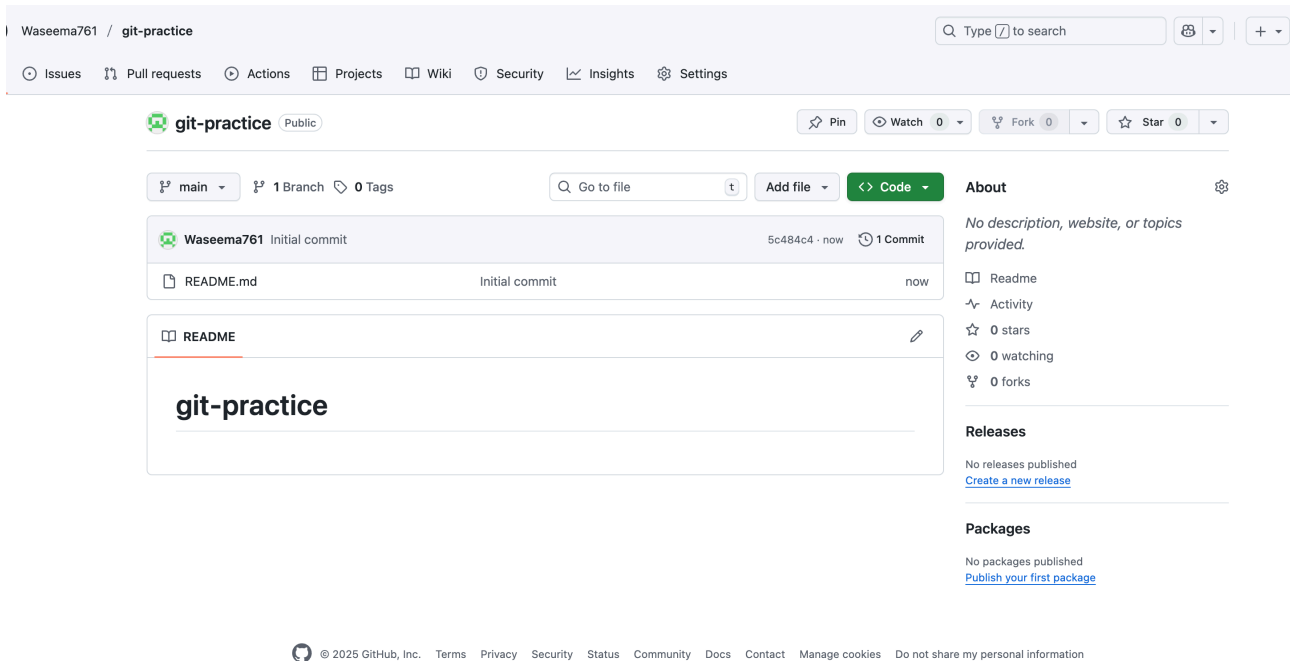
Add license

Licenses explain how others can use your code. [About licenses](#)

No license ▾

Create repository

—> Given name **git-practice** as repository name
—> add README ON
—> no .gitignore



3) Clone the created repo to local.

Copy the repository link

And

Use git clone (paste the url) it will create a clone of your repository

```
waseemakram@waseem ~ % cd myproject.git
[waseemakram@waseem myproject.git % git clone git@github.com:Waseema761/git-practice.git
Cloning into 'git-practice'...
[Enter passphrase for key '/Users/waseemakram/.ssh/id_ed25519':
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
[waseemakram@waseem myproject.git % ls
git-practice
waseemakram@waseem myproject.git %
```

Now check the clone directory and Ls you will find same data of repository

```
[waseemakram@waseem myproject.git % ls
git-practice
[waseemakram@waseem myproject.git % cd git-practice
[waseemakram@waseem git-practice % ls
README.md
waseemakram@waseem git-practice %
```

- 4) Create two files in local repo.
- 5) Commit two files and push to central Repository.

Created two files and
Git add . For adding all files in repo
Git commit -m for committing changes

```
file1      file2      README.md
[waseemakram@waseem git-practice % git add .
[waseemakram@waseem git-practice % git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   file1
        new file:   file2

[waseemakram@waseem git-practice % git commit -m " added two files "
[main eb21d47] added two files
 2 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 file1
 create mode 100644 file2
[waseemakram@waseem git-practice % git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
```

Git push -u origin main will push that file to central repo

```
file1      file2      README.md
waseemakram@waseem git-practice % git push -u origin main
Enter passphrase for key '/Users/waseemakram/.ssh/id_ed25519':
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 283 bytes | 283.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:Waseema761/git-practice.git
   5c484c4..eb21d47  main -> main
branch 'main' set up to track 'origin/main'.
waseemakram@waseem git-practice %
```

Files are now available in central repository

The screenshot shows a GitHub repository page for 'git-practice' by user 'Waseema761'. The repository is public and has 1 branch (main) and 0 tags. The commit history shows two commits: 'Initial commit' (11 minutes ago) and 'added two files' (4 minutes ago). The commit 'added two files' includes files 'file1' and 'file2'. The README file is visible, titled 'git-practice'. The right sidebar shows the repository's 'About' section, which is currently empty, and the 'Releases' and 'Packages' sections, both showing no published items.

6) Create a branch in local and create a sample file and push to central.

```
[waseemakram@waseem git-practice % git branch release
[waseemakram@waseem git-practice % git branch --list
* main
  release
[waseemakram@waseem git-practice % ls
file1      file2      README.md
[waseemakram@waseem git-practice % git checkout release
Switched to branch 'release'
[waseemakram@waseem git-practice % ls
file1      file2      README.md
```

—> git branch release

This command will create a branch name release

—> git branch —list

This command is used for checking branch list

—> git checkout release

This command will switch to the release named branch

* when we move to other other branch it will default save the files of previous branch as we can see we are having files of main in release branch

We created a file name world in release branch

And we pushed that branch using

—> Git push -u origin release

```
[waseemakram@waseem git-practice % vi world
[waseemakram@waseem git-practice % git status
On branch release
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        world

nothing added to commit but untracked files present (use "git add" to track)
[waseemakram@waseem git-practice % git add .
[waseemakram@waseem git-practice % git status
On branch release
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   world

[waseemakram@waseem git-practice % git commit -m "added file world"
[release 3266632] added file world
 1 file changed, 1 insertion(+)
 create mode 100644 world
[waseemakram@waseem git-practice % git status
On branch release
nothing to commit, working tree clean
```

```
[waseemakram@waseem git-practice % git push -u origin release
[Enter passphrase for key '/Users/waseemakram/.ssh/id_ed25519':
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
```

We pushed the branch to central repo

We can see release named branch is available in central repo

 **git-practice** Public

 Pin  Watch 0

 **release** had recent pushes 1 minute ago Compare & pull request

 **main**  2 Branches  0 Tags

t Add file <> Code

Switch branches/tags

Branches

Tags

✓ main default

release

[View all branches](#)

eb21d47 · 16 hours ago 2 Commits

Initial commit

17 hours ago

added two files

16 hours ago

added two files

16 hours ago

git-practice

Release branch

 **release** had recent pushes 1 minute ago Compare & pull request

 **release**  2 Branches  0 Tags

t Add file <> Code

This branch is 1 commit ahead of **main** Contribute

 **Waseema761** added file world 3266632 · 2 minutes ago 3 Commits

README.md

Initial commit

17 hours ago

file1

added two files

16 hours ago

file2

added two files

16 hours ago

world

added file world

2 minutes ago

 **README**

git-practice

Branches list in central repository

Search branches...					
Default					
Branch	Updated	Check status	Behind	Ahead	Pull request
main	16 hours ago				Default
Your branches					
Branch	Updated	Check status	Behind	Ahead	Pull request
waseem	now		0	0	
release	6 minutes ago		0	1	
Active branches					
Branch	Updated	Check status	Behind	Ahead	Pull request
waseem	now		0	0	
release	6 minutes ago		0	1	

7) Create a branch in github and clone that to local.

We created a branch in our central repository

```
branch 'release' set up to track 'origin/release' .
[waseemakram@waseem git-practice % git clone -b waseem git@github.com:Waseema761/git-practice.git
Cloning into 'git-practice'...
[Enter passphrase for key '/Users/waseemakram/.ssh/id_ed25519':
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 9 (delta 1), reused 6 (delta 1), pack-reused 0 (from 0)
Receiving objects: 100% (9/9), done.
Resolving deltas: 100% (1/1), done.
waseemakram@waseem git-practice %
```

—> used git clone waseem (ssh url)
It will copy the waseem branch from cenral repo

```
Resolving deltas: 100% (1/1), done.
[waseemakram@waseem git-practice % ls
file1      file2      git-practice  README.md    world
[waseemakram@waseem git-practice % cd git-practice
[waseemakram@waseem git-practice % ls
file1      file2      README.md
waseemakram@waseem git-practice %
```

We can see waseem name branch is active in clone

```
[waseemakram@waseem git-practice % cd git-practice
[waseemakram@waseem git-practice % ls
file1      file2      README.md
[waseemakram@waseem git-practice % git branch
* waseem
waseemakram@waseem git-practice %
```

8) Merge the created branch with master in git local.

—> we are in a main branch we merged the waseem named branch to main branch

—> git merge waseem

It will merge the data to present branch

```
[waseemakram@waseem git-practice % ls
file1          file2          README.md      was
[waseemakram@waseem git-practice % git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
[waseemakram@waseem git-practice % ls
file1          file2          README.md
[waseemakram@waseem git-practice % git merge waseem
Updating eb21d47..3b76fff
Fast-forward
 was | 0
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 was
[waseemakram@waseem git-practice % ls
file1          file2          README.md      was
waseemakram@waseem git-practice %
```

—> we can see was names file is merged to main branch from waseem branch

9) Merge the created branch with master in github by sending a pull request.

“Compare & pull request” for waseem → click it.

Base branch: main

Compare branch: waseem

Click Create pull request.

base: main ← compare: waseem ✓ Able to merge. These branches can be automatically merged.

Add a title

creating pull request

Add a description

Write Preview H B I `< >`

hello

Markdown is supported Paste, drop, or click to add files

Create pull request

Remember, contributions to this repository should follow our [GitHub Community Guidelines](#).

Helpful resources

[GitHub Community Guidelines](#)

1 commit 1 file changed 1 contributor

Conversation 0

Commits 1

Checks 0

Files changed 1

Waseema761 commented now

Owner ***

hello

😊

added akram in waseem

a825ab0

✓ No conflicts with base branch

Merging can be performed automatically.

Merge pull request

You can also merge this with the command line. [View command line instructions.](#)

Still in progress? [Convert to draft](#)

Add a comment

Write

Preview

🔗

H

B

I

☰

<>

🔗

☰

☰

☰

🖋

@

💬

↩

📄

Add your comment here...

Reviewers

No reviews

Still in progress? [Convert to draft](#)

Assignees

No one—[assign yourself](#)

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

None yet

Confirm the pull request



Commit message

Extended description

creating pull request

This commit will be authored by waseem.akram082003@gmail.com.

Confirm mergeCancel

Successfully merged the branch in central repository

The screenshot displays a GitHub pull request interface. At the top, a purple banner indicates 'Merged' with the text 'Waseema761 merged 1 commit into main from waseem'. Below this, a summary bar shows 'Conversation 0', 'Commits 1', 'Checks 0', and 'Files changed 1'. The main content area shows a comment from 'Waseema761' saying 'hello' with a smiley face emoji. Below the comment, a commit history shows 'added akram in waseem' and 'Waseema761 merged commit f45129d into main now'. A 'Revert' button is visible. A message states 'Pull request successfully merged and closed' with a 'Delete branch' button. At the bottom, there is a 'Add a comment' section with a text area and a 'Comment' button. On the right side, there are sections for 'Reviewers' (No reviews), 'Assignees' (No one—assign yourself), 'Labels' (None yet), 'Projects' (None yet), 'Milestone' (No milestone), 'Development' (Successfully merging this these issues. None yet), and 'Notifications' (You're receiving notificati the open/close state. Uns).

—>We can see akram named file is available in main which was from waseem branch

The screenshot shows a GitHub repository page for 'git-practice' (Public). The repository has 3 branches and 0 tags. A table lists the commit history:

File	Commit Message	Time
README.md	Initial commit	18 hours ago
akram	added akram in waseem	4 minutes ago
file1	added two files	18 hours ago
file2	added two files	18 hours ago
was	added was file in waseem	32 minutes ago

Below the table, the 'README' file is shown with the title 'git-practice'. On the right side, there are sections for 'About' (No description, website, or topics provided.), 'Releases' (No releases published, Create a new release), and 'Packages' (No packages published, Publish your first package).

10) Create a file in local and send that to branch in github.

```
waseemakram@waseem git-practice % touch akram
waseemakram@waseem git-practice % git add .
waseemakram@waseem git-practice % git commit -m " added akram in waseem"
[waseem a825ab0] added akram in waseem
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 akram
waseemakram@waseem git-practice % ls
akram      file1      file2      README.md  was
waseemakram@waseem git-practice % git push -u origin waseem

[Enter passphrase for key '/Users/waseemakram/.ssh/id_ed25519':
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 239 bytes | 239.00 KiB/s, done.
Total 2 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:Waseema761/git-practice.git
3b76fff..a825ab0 waseem -> waseem
branch 'waseem' set up to track 'origin/waseem'.
```

- > Created a file using touch
- > git add . (will add that file for local repo)
- > git commit -m (will commit the changes)
- > git push -u origin waseem (pushes the file from local repo to central repo)

Check central repo of waseem branch

waseem 3 Branches 0 Tags

Go to file t Add file <> Code

This branch is 1 commit behind main . Contribute

Waseema761 added akram in waseem a825ab0 · 7 minutes ago 4 Commits

README.md	Initial commit	18 hours ago
akram	added akram in waseem	7 minutes ago
file1	added two files	18 hours ago
file2	added two files	18 hours ago
was	added was file in waseem	35 minutes ago

README

git-practice

11) Clone only a branch from github to local.

—> Git clone -b waseem (ssh url)
-b waseem (it will clone waseem named branch from repository)

waseem had recent pushes 18 seconds ago [Compare & pull request](#)

waseem 3 Branches 0 Tags [Add file](#) [Code](#)

This branch is 1 commit ahead of and 1 commit behind main. [Contribute](#)

Commit	Message	Time
098d0a5	Create file.clone	1 minute ago
	added akram in waseem	1 hour ago
	added two files	20 hours ago
	added two files	20 hours ago
	added was file in waseem	2 hours ago
	Initial commit	20 hours ago

git-practice

—> Now check in your local repo

```
waseemakram@waseem myproject.git % git clone -b waseem git@github.com:Waseema761/git-practice.git
Cloning into 'git-practice'...
[Enter passphrase for key '/Users/waseemakram/.ssh/id_ed25519':
remote: Enumerating objects: 17, done.
remote: Counting objects: 100% (17/17), done.
remote: Compressing objects: 100% (10/10), done.
remote: Total 17 (delta 6), reused 6 (delta 2), pack-reused 0 (from 0)
Receiving objects: 100% (17/17), done.
Resolving deltas: 100% (6/6), done.
waseemakram@waseem myproject.git % ls
akram      file1      file2      git-practice  README.md  was
waseemakram@waseem myproject.git % cd git-practice
waseemakram@waseem git-practice % ls
akram      file.clone  file1      file2      README.md  was
waseemakram@waseem git-practice %
```


12) Create a file with all passwords and make that untrackable with git.

—> Touch password.txt (created a file)
Opened hidden file for untrackable with git
—> Vi .gitignore
Add the password.txt file inside vi .gitignore

```
waseem
[waseemakram@waseem myproject.git % touch passwords.txt
[waseemakram@waseem myproject.git % vi passwords.txt
[waseemakram@waseem myproject.git % vi .gitignore
[waseemakram@waseem myproject.git % git add .
[waseemakram@waseem myproject.git % git commit -m "igone password file"
[waseem 02279ae] igone password file
1 file changed, 1 insertion(+)
create mode 100644 .gitignore
```

```
[waseemakram@waseem myproject.git % ls
akram      file1      file2      git-practice  passwords.txt  README.md    was
[waseemakram@waseem myproject.git % cat .gitignore
passwords.txt
[waseemakram@waseem myproject.git %
```

We confirm it by checking git status
We can't see password.txt file in status as it is now untraceable

```
passwords.txt
[waseemakram@waseem myproject.git % git status
On branch waseem
Your branch is ahead of 'origin/waseem' by 2 commits.
(use "git push" to publish your local commits)

nothing to commit, working tree clean
waseemakram@waseem myproject.git %
```

We can check in central repo

 waseema/01 committed 20 minutes ago

igone password file

 waseem

1 parent [4747087](#) comm

Q Filter files...

 .gitignore

 1 file changed +1 -0 lines changed

Q Search within code

▼ .gitignore 

```
... @@ -0,0 +1 @@
1 + passwords.txt
```

Comments 0

 Lock conversation

 Comment

 Subscribe You're not receiving notifications from this thread.

13) Make a commit and make that commit reset without saving changes.

We created a file and

—> git add .

—> git commit -m

—> git log (it will check all the previous commits)

```
[waseemakram@waseem myproject.git % touch file3
[waseemakram@waseem myproject.git % git add .
[waseemakram@waseem myproject.git % git commit -m "added the file3 "
[waseem f93f296] added the file3
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 file3
[waseemakram@waseem myproject.git % git log
commit f93f2965369e9c2488ba398dbd5d8fb9fe4ef4ea (HEAD -> waseem)
Author: waseem_akram <waseem.akram082003@gmail.com>
Date:   Fri Nov 14 15:25:01 2025 +0530

    added the file3

commit 02279ae6cd6109109e29c267cf29c53936280004
Author: waseem_akram <waseem.akram082003@gmail.com>
Date:   Fri Nov 14 15:17:10 2025 +0530

    igone password file
```

Now git reset —hard HEAD~1 (this will reset the head commit)

Now check the git log again the file is gone

```
waseemakram@waseem myproject.git % git reset --hard HEAD~1

HEAD is now at 02279ae igone password file
[waseemakram@waseem myproject.git % git log
commit 02279ae6cd6109109e29c267cf29c53936280004 (HEAD -> waseem)
Author: waseem_akram <waseem.akram082003@gmail.com>
Date:   Fri Nov 14 15:17:10 2025 +0530

    igone password file

commit 4747087fd07a0378fbee19a5a76351760031b856
```

14) Revert a committed commit to the older version.

—>Rm akram (removed the file)
—> git add .
—> git commit -m
For changes

```
[waseemakram@waseem myproject.git % rm akram
[waseemakram@waseem myproject.git % git add .
[waseemakram@waseem myproject.git % git status
On branch waseem
Your branch is up to date with 'origin/waseem'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        deleted:    akram

[waseemakram@waseem myproject.git % git commit -m "deleted file "
[waseem ebd26e5] deleted file
1 file changed, 0 insertions(+), 0 deletions(-)
delete mode 100644 akram
```

—> Git log for commits list
Now copy the recently deleted file commit

```
nothing to commit, working tree clean
[waseemakram@waseem myproject.git % git log
commit ebd26e5b8c19473eee1d1022e3c599136a42c279 (HEAD -> waseem)
Author: waseem_akram <waseem.akram082003@gmail.com>
Date:   Fri Nov 14 17:38:59 2025 +0530

    deleted file
```

And use

—>Git revert (commit add here) this will revert that commit

```
[waseemakram@waseem myproject.git % ls
file1      file2      git-practice  passwords.txt  README.md    was
[waseemakram@waseem myproject.git % git revert ebd26e
[waseem 1373586] Revert "deleted file"
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 akram
[waseemakram@waseem myproject.git % ls
akram      file1      file2      git-practice  passwords.txt  README.md    was
[waseemakram@waseem myproject.git % $
```

```
Revert "deleted file"
```

```
This reverts commit ebd26e5b8c19473eee1d1022e3c599136a42c279.
```

```
# Please enter the commit message for your changes. Lines starting  
# with '#' will be ignored, and an empty message aborts the commit.  
#
```

```
# On branch waseem
```

```
# Your branch is ahead of 'origin/waseem' by 2 commits.
```

```
# (use "git push" to publish your local commits)
```

```
#
```

```
# Changes to be committed:
```

```
#       new file:   akram
```

```
#
```

```
# Untracked files:
```

```
#       passwords.txt
```

```
#
```

```
~
```

```
~
```

```
~
```

```
~
```

```
~
```

```
~
```

```
~
```

```
~
```

```
~
```

```
~
```

```
~
```

```
~
```

The file is reverted successfully

15) Push a file to stash without saving the changes and work on another file.

- > Vi alphabet created a file with content
- > git add .
- > git status to check

```
[waseemakram@waseem myproject.git % vi alphabet
[waseemakram@waseem myproject.git % cat alphabet
hello
[waseemakram@waseem myproject.git % git add .
[waseemakram@waseem myproject.git % git status
On branch waseem
Your branch is ahead of 'origin/waseem' by 5 commits.
(use "git push" to publish your local commits)

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   alphabet
```

Now if we want to work on another file keeping this side
We can use --> git stash
It will save the work
And we can start working on another file

```
[waseemakram@waseem myproject.git % git stash
Saved working directory and index state WIP on waseem: 4d6e20e was
```

Like I have started working on hello file

```
waseemakram@waseem myproject.git % git stash
Saved working directory and index state WIP on waseem: 4d6e20e was
waseemakram@waseem myproject.git % ls
akram      file1      file2      git-practice  passwords.txt  README.md    was
waseemakram@waseem myproject.git % vi hello
waseemakram@waseem myproject.git % git add .
waseemakram@waseem myproject.git % git commit -m "hello file"
[waseem b5891d9] hello file
1 file changed, 1 insertion(+)
create mode 100644 hello
waseemakram@waseem myproject.git % ls
akram      file1      file2      git-practice  hello          passwords.txt  README.md    was
waseemakram@waseem myproject.git % git pop
```

16) Undo the stash file and start working on that again.

And later now I want to continue my previous unfinished work of alphabet file
We can use —> git stash pop

- > git stash list (it will list the stash files)
- > git stash pop (it will continue the file again)

```

Log
[waseemakram@waseem myproject.git % git stash list
stash@{0}: WIP on waseem: 4d6e20e was
[waseemakram@waseem myproject.git % git stash pop
On branch waseem
Your branch is ahead of 'origin/waseem' by 7 commits.
  (use "git push" to publish your local commits)

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   alphabet

Dropped refs/stash@{0} (605500856a966d443eb7b1bbc60be24bd8a15cbb)
waseemakram@waseem myproject.git %

```

17)Generate a ssh-keygen and configure into github.

- >Ssh-keygen
- Will create a ssh key
- Now copy the pub key by cat command

```

[waseemakram@waseem myproject.git % ssh-keygen
Generating public/private ed25519 key pair.
Enter file in which to save the key (/Users/waseemakram/.ssh/id_ed25519): /Users/waseemakram/.ssh/id_ed25519
/Users/waseemakram/.ssh/id_ed25519 already exists.
Overwrite (y/n)? y
[Enter passphrase for "/Users/waseemakram/.ssh/id_ed25519" (empty for no passphrase):
[Enter same passphrase again:
Your identification has been saved in /Users/waseemakram/.ssh/id_ed25519
Your public key has been saved in /Users/waseemakram/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:uQEMzq+1giYJyG4XlNnCIHKCzeVwUJUYybApgcYJ0sI waseemakram@waseem
The key's randomart image is:
+--[ED25519 256]--+
|O=*o*o==.      |
|B*=O+*.        |
|E.**.o          |
|+..... . .    |
|o. . o S       |
|o. ..o . o     |
|ooo.o . .      |
|o. .           |
|               |
+-----[SHA256]-----+

```

Copy the pub key from ssh to waseemakram@waseem

```

[waseemakram@waseem myproject.git % cat /Users/waseemakram/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIOIML+6Xxi4cOYSNAS1+7SNCSHQ8+8twgBJorbKSV4KE waseemakram@waseem
waseemakram@waseem myproject.git %

```

We can delete the key can create a new key by simply clicking new ssh key and past the pub key which you copied from terminal and save it

Waseem (waseemakram)Personal account

Go to your personal profile

SSH keys

New SSH key

This is a list of SSH keys associated with your account. Remove any keys that you do not recognize.

Authentication keys

my_mac

SHA256:AvXmyqvKtCOP/ErIUwLBmSGhACGue08m3U10VcEXS+s

Added on Oct 22, 2025

Last used within the last week — Read/write

Delete

Check out our guide to [connecting to GitHub using SSH keys](#) or troubleshoot [common SSH problems](#).

GPG keys

New GPG key

There are no GPG keys associated with your account.

Learn how to [generate a GPG key and add it to your account](#).

19) Basic understanding of .git file.

—> when we run git init . It will create a hidden .git folder

Every Git repository has a hidden folder called .git.
We can confirm by ls -a

```
ssh-ed25519 AAAAC3NzaC1lZDI1IEN5AAAAIO1ML+6XX14C0
waseemakram@waseem myproject.git % ls -a
.  ..  .git
waseemakram@waseem myproject.git %
```

—> .git contains **all the information Git needs** to track your project,

```
waseemakram@waseem .git % ls
AUTO_MERGE  COMMIT_EDITMSG  config          description     FETCH_HEAD     HEAD           hooks          index          info          logs          objects        OR
waseemakram@waseem .git %
```

—> Everything Git does (commits, merges, reverts, stash) is stored in this folder.

What's inside .git (in simple words)

1. HEAD

- A small file that tells Git **which branch you are currently on**.
- Example: `master`, `main`, `waseem`.

2. config

- Contains **your repository settings**:
 - `username`
 - `email`
 - `remote origin URL`

3. branches/

- (Rarely used now)
- Stores info about old branch pointers.

4. refs/

This folder contains **pointers to your commits**.

Inside it:

a) refs/heads/

- Stores the **latest commit ID for each branch**.
- **Example:**

main

- `waseem`
- `release`

b) refs/tags/

- Stores pointers for **tags**.

5. info/

- Contains the **exclude** file (like a local **.gitignore**).

6. objects/

This is the **most important** part.

It stores all actual Git data:

- your commits
- your files (in compressed form)
- your trees (folders)
- your blobs (file data)

Every commit you make is stored as an **object** inside this folder.

Important note of **.git**

Do **not delete** the **.git** folder.

If you delete it → your project becomes a **normal folder**, and all Git history is LOST.

20)Check all the logs of git.

—> git log (command use for checking all logs)

Shows commit history for the current branch

```
waseemakram@waseem .git % git log
commit b5891d9078b5be047803068147924ef969705e68 (HEAD -> waseem)
Author: waseem_akram <waseem.akram082003@gmail.com>
Date: Fri Nov 14 18:24:38 2025 +0530

    hello file

commit 4d6e20eb818273400f677f1e07c172c9920ab0a7
Author: waseem_akram <waseem.akram082003@gmail.com>
Date: Fri Nov 14 18:20:54 2025 +0530

    was

commit 21560e085f5b8c2ecadf0801fb815616685290ee
Author: waseem_akram <waseem.akram082003@gmail.com>
Date: Fri Nov 14 18:04:14 2025 +0530

    deleted alhabet

commit 99365b1bc36b4407df76d1328798dfb8fc6572eb
Author: waseem_akram <waseem.akram082003@gmail.com>
Date: Fri Nov 14 18:01:11 2025 +0530

    added

commit 1373586c11a039a1986c0ed818fef21eade94406
Author: waseem_akram <waseem.akram082003@gmail.com>
Date: Fri Nov 14 17:41:31 2025 +0530

    Revert "deleted file"

    This reverts commit ebd26e5b8c19473eee1d1022e3c599136a42c279.

commit 214a41f6679a6f2e040ac6eb030af205a5dad96
Author: waseem_akram <waseem.akram082003@gmail.com>
Date: Fri Nov 14 17:40:13 2025 +0530

    Revert "igone password file"

    This reverts commit 02279ae6cd6109109e29c267cf29c53936280004.

commit ebd26e5b8c19473eee1d1022e3c599136a42c279
Author: waseem_akram <waseem.akram082003@gmail.com>
Date: Fri Nov 14 17:38:59 2025 +0530

    deleted file

commit 02279ae6cd6109109e29c267cf29c53936280004 (origin/waseem)
Author: waseem_akram <waseem.akram082003@gmail.com>
Date: Fri Nov 14 15:17:10 2025 +0530

    igone password file

commit 4747087fd07a0378fbee19a5a76351760031b856
Author: waseem_akram <waseem.akram082003@gmail.com>
Date: Fri Nov 14 15:05:19 2025 +0530

    added clone file

commit a825ab07025deaf4dbffd1eb9ca4787fafd19790
```

21)Rename a commit message

We can edit commit by
—> git commit —amend

```
[waseemakram@waseem myproject.git % git commit --amend
[waseem 78d30d3] changed the commit message
Date: Fri Nov 14 18:24:38 2025 +0530
2 files changed, 2 insertions(+)
create mode 100644 alphabet
create mode 100644 hello
[waseemakram@waseem myproject.git % git log -3
```

It will open the commit
We can insert and rename
—> but it is not recommended to use

changed the commit message

```
# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# Date:      Fri Nov 14 18:24:38 2025 +0530
#
# On branch waseem
# Your branch is ahead of 'origin/waseem' by 7 commits.
#   (use "git push" to publish your local commits)
#
# Changes to be committed:
#   new file:   alphabet
#   new file:   hello
#
~
~
~
~
~
~
~
```

```
create mode 100644 hello
[waseemakram@waseem myproject.git % git log -3
commit 78d30d39543cd917339fe15015c8871d31b85eb1 (HEAD -> waseem)
Author: waseem_akram <waseem.akram082003@gmail.com>
Date:   Fri Nov 14 18:24:38 2025 +0530

    changed the commit message

commit 4d6e20eb818273400f677f1e07c172c9920ab0a7
Author: waseem_akram <waseem.akram082003@gmail.com>
Date:   Fri Nov 14 18:20:54 2025 +0530
```

22) Merge multiple commits into a single commit (squash)

→ git rebase -i HEAD~3
Will merge 3 commits from HEAD

```
waseemakram@waseem myproject.git % git rebase -i HEAD~3
[detached HEAD f3271d8] deleted alhabet
Date: Fri Nov 14 18:04:14 2025 +0530
3 files changed, 3 insertions(+)
create mode 100644 hello
Successfully rebased and updated refs/heads/waseem.
```

It will open this editor where we can squash

```
# pick 21560e0 # deleted alhabet
pick 4d6e20e # was
pick 78d30d3 # changed the commit message

# Rebase 99365b1..78d30d3 onto 99365b1 (3 commands)
#
# Commands:
# p, pick <commit> = use commit
# r, reword <commit> = use commit, but edit the commit message
# e, edit <commit> = use commit, but stop for amending
# s, squash <commit> = use commit, but meld into previous commit
# f, fixup [-C | -c] <commit> = like "squash" but keep only the previous
#                               commit's log message, unless -C is used, in which case
#                               keep only this commit's message; -c is same as -C but
#                               opens the editor
# x, exec <command> = run command (the rest of the line) using shell
# b, break = stop here (continue rebase later with 'git rebase --continue')
# d, drop <commit> = remove commit
# l, label <label> = label current HEAD with a name
# t, reset <label> = reset HEAD to a label
# m, merge [-C <commit> | -c <commit>] <label> [# <oneline>]
#     create a merge commit using the original merge commit's
#     message (or the oneline, if no original merge commit was
#     specified); use -c <commit> to reword the commit message
# u, update-ref <ref> = track a placeholder for the <ref> to be updated
#                       to this position in the new commits. The <ref> is
#                       updated at the end of the rebase
#
# These lines can be re-ordered; they are executed from top to bottom.
#
# If you remove a line here THAT COMMIT WILL BE LOST.
#
# However, if you remove everything, the rebase will be aborted.
#
~
~
~
~
~
~
~
~
~
~
~
```

```

pick 21560e0 # deleted alhabet
squash 4d6e20e # was
squash 78d30d3 # changed the commit message

# Rebase 99365b1..78d30d3 onto 99365b1 (3 commands)
#
# Commands:
# p, pick <commit> = use commit
# r, reword <commit> = use commit, but edit the commit message
# e, edit <commit> = use commit, but stop for amending
# s, squash <commit> = use commit, but meld into previous commit
# f, fixup [-C | -c] <commit> = like "squash" but keep only the previous
#                               commit's log message, unless -C is used, in which case
#                               keep only this commit's message; -c is same as -C but
#                               opens the editor
# x, exec <command> = run command (the rest of the line) using shell
# b, break = stop here (continue rebase later with 'git rebase --continue')
# d, drop <commit> = remove commit
# l, label <label> = label current HEAD with a name
# t, reset <label> = reset HEAD to a label
# m, merge [-C <commit> | -c <commit>] <label> [# <oneline>]
#       create a merge commit using the original merge commit's
#       message (or the oneline, if no original merge commit was
#       specified); use -c <commit> to reword the commit message
# u, update-ref <ref> = track a placeholder for the <ref> to be updated
#                       to this position in the new commits. The <ref> is
#                       updated at the end of the rebase
#
# These lines can be re-ordered; they are executed from top to bottom.
#
# If you remove a line here THAT COMMIT WILL BE LOST.
#
# However, if you remove everything, the rebase will be aborted.
#
~

```

Now esc
And
:wq!

Now we check

—> git log

It has merged three commits in single commit

```
waseemakram@waseem myproject.git % git rebase -i HEAD~3
[detached HEAD f3271d8] deleted alhabet
Date: Fri Nov 14 18:04:14 2025 +0530
3 files changed, 3 insertions(+)
create mode 100644 hello
Successfully rebased and updated refs/heads/waseem.
waseemakram@waseem myproject.git % git log
commit f3271d8cf068a360b10bc8efefd593c871eec139 (HEAD -> waseem)
Author: waseem_akram <waseem.akram082003@gmail.com>
Date: Fri Nov 14 18:04:14 2025 +0530

    deleted alhabet

    was

    changed the commit message

commit 99365b1bc36b4407df76d1328798dfb8fc6572eb
Author: waseem_akram <waseem.akram082003@gmail.com>
Date: Fri Nov 14 18:01:11 2025 +0530

    added
```